Finance Ledger & Reconciliation Microservice

A Production-Grade Finance API built with Spring Boot 3, PostgreSQL, JWT Security and Docker

Presented by

Theophilus Danladi Umar

Flexisaf Internship Program • April 2026

Problem Statement

Manual Finance Tracking	School bursary systems and fee portals rely on error-prone manual processes to track debits, credits, and balances.
Duplicate Payments	Without idempotency controls, payment gateways can charge the same invoice multiple times, leading to financial loss.
No Audit Trail	There is no tamper-evident record of who performed what financial action and when.
Reconciliation Gaps	Matching bank statement entries to system payments is done manually, missing anomalies like outliers and negative balances.

Solution Overview

A production-ready REST microservice that automates the full finance lifecycle — from ledger accounting to invoice payment and bank reconciliation.

Double-Entry Ledger	Every transaction creates balanced DEBIT/CREDIT journal entries
Invoice & Payment Flow	Full invoice lifecycle with idempotent payment capture
PDF Receipt Generation	Thymeleaf-rendered receipts downloadable as PDF
Bank Reconciliation	Import statements and auto-match against payments
Anomaly Detection	Flags duplicates, negative balances and outlier payments
Audit Trail	Every action is logged with user, entity and metadata

Technology Stack

Core	Java 17	Spring Boot 3	Spring Web	Spring Validation
Data	PostgreSQL 15	Spring Data JPA	Hibernate	Flyway Migrations
Security	Spring Security	JWT (JJWT 0.12.x)	HMAC Webhook	BCrypt
Docs & PDF	OpenAPI/Swagger	Thymeleaf	Flying Saucer PDF	
Testing	JUnit 5	Mockito	AssertJ	
DevOps	Docker	Docker Compose	Render (Cloud)	GitHub

System Architecture & Data Model

The system is built around 9 core entities:

Users	Top-level entity. Owns accounts and generates audit logs.
Accounts	Financial ledger accounts (ASSET, LIABILITY, EQUITY, REVENUE, EXPENSE).
Journal Entries	DEBIT/CREDIT movements forming the double-entry ledger.
Invoices	Amounts owed, with idempotency key to prevent duplicates.
Payments	Settles invoices. Idempotent. Tracks gateway references.
Receipts	PDF documents generated on invoice payment.
Bank Statements	Imported bank files with opening/closing balances.
Reconciliation Items	Links payments to statements. Flags MATCHED/ANOMALY.
Audit Logs	Full tamper-evident trail with JSONB metadata.

Key Features

Double-Entry Bookkeeping	Every financial movement creates balanced DEBIT and CREDIT journal entries ensuring the ledger always balances.
Idempotent Payment Capture	Payments are deduplicated using idempotency keys — submitting the same payment twice returns the original result safely.
Invoice Lifecycle Management	Full invoice flow: PENDING → PARTIALLY_PAID → PAID → OVERDUE → CANCELLED with automatic status updates.
PDF Receipt Generation	Thymeleaf-rendered HTML receipts are converted to downloadable PDF files using Flying Saucer.
Bank Reconciliation Engine	Import bank statements and automatically match completed payments, flagging unmatched and anomalous entries.
JWT + HMAC Security	Stateless JWT authentication for API access and HMAC-SHA256 signature verification for webhook payloads.

API Endpoints

Method	Endpoint	Description
POST	`/api/auth/register`	Register a new user
POST	`/api/auth/login`	Login and receive JWT token
POST	`/api/accounts`	Create a ledger account
GET	`/api/accounts`	Get all accounts for user
POST	`/api/journal`	Create a journal entry
GET	`/api/journal/trial-balance`	Get trial balance report
POST	`/api/invoices`	Create an invoice
PATCH	`/api/invoices/{id}/status`	Update invoice status
POST	`/api/payments`	Capture a payment (idempotent)
GET	`/api/receipts/{id}/download`	Download PDF receipt
POST	`/api/reconciliation/import`	Import bank statement
POST	`/api/reconciliation/{id}/reconcile`	Run reconciliation
GET	`/api/audit/me`	Get my audit logs

Anomaly Detection Engine

The reconciliation engine automatically flags suspicious payment patterns by applying three rule-based checks against every completed payment:

Rule 1	NEGATIVE BALANCE	Any payment with an amount of zero or below is immediately flagged. This catches data integrity issues and malformed gateway responses.
Rule 2	DUPLICATE PAYMENT	If two or more payments share the same invoice ID and the same amount, the system flags all subsequent ones as duplicates to prevent double-charging.
Rule 3	OUTLIER PAYMENT	Any payment that exceeds 120% of the invoice total is flagged as an outlier. This catches over-payments and potential fraud attempts.

Challenges & Learnings

JJWT 0.12.x API Breaking Changes	The JJWT library changed its API significantly in version 0.12.x — parserBuilder() became parser(), getBody() became getPayload(), and Key became SecretKey. Resolved by reading the migration guide and updating all affected methods.
Docker Compose over Local PostgreSQL	IntelliJ had persistent connection issues with a locally installed PostgreSQL. Switching to Docker Compose with explicit 127.0.0.1 binding resolved all connection failures and made the setup fully reproducible.
Idempotency Key Pattern	Implementing safe payment retries required understanding the idempotency key pattern — checking for existing records before creating new ones, ensuring duplicate submissions return the original result without side effects.
Flyway Migration Ordering	Foreign key constraints required strict migration ordering. V1 (users) must run before V2 (accounts) which must run before V4 (invoices). Any ordering mistake causes the app to fail on startup.
Production Environment Variables	Hardcoded credentials were replaced with environment variables for Render deployment. Spring profiles (default vs prod) allowed different configs for local and live environments.

Live Demo & Links

Live API	`https://finance-ledger-service.onrender.com`
Swagger UI	`https://finance-ledger-service.onrender.com/swagger-ui/index.html`
GitHub Repo	`https://github.com/Theophilus131/finance-ledger-service`

Thank you for the opportunity to build this project. It was a great learning experience working through real-world finance engineering challenges at Flexisaf.

Theophilus Danladi Umar • Flexisaf Internship • April 2026